ATLAS

Presentazione Architettura

Gruppo 9 - A.A. 2025-2026

Francesco Marcolongo
Federico Simonetto
Andrea Difino
Michele Tesser
Bilal Sabic
Giacomo Giora
Riccardo Valerio

Automated EN18031 Compliance Verification

Capitolato C1 : Automated EN18031 Compliance Verification

Obiettivo: Sviluppare un’applicazione web che automatizzi e guidi la verifica di conformità dei dispositivi radio allo standard EN 18031, che definisce dei requisiti di cybersicurezza.
Ciò viene fatto eseguendo i decision tree dei requisiti in modo strutturato e tracciabile.

Modello Client-Server

- **Separazione delle Responsabilità**: Netta divisione tra l'interfaccia utente (Client) e il motore di elaborazione dati (Server) per garantire massima modularità e disaccoppiamento.

  Client / Frontend (React SPA) $\rightleftharpoons$ Server / Backend (Python + FastAPI)

- **Comunicazione Disaccoppiata (RESTful API):** I due moduli dialogano in modo indipendente tramite protocollo HTTP/REST.

Architettura di deployment: due monoliti

- Contesto aziendale interno con utenza limitata: la complessità dei microservizi sarebbe overhead puro
- Backend (FastAPI) e Frontend (React) come container separati, comunicanti tramite HTTP
- Deploy indipendente e semplificato tramite Docker Compose

Perchè due monoliti?

- **Separazione netta delle responsabilità**: il Frontend gestisce la presentazione, il Backend la logica di business e la persistenza
- **Deploy indipendente**: è possibile aggiornare o sostituire uno dei due senza toccare l'altro

Architettura logica: Layered

PRESENTATION LAYER → **APPLICATION LAYER** → **DOMAIN LAYER** → **INFRASTRUCTURE LAYER**

Gestisce la comunicazione con il frontend

Orchestra gli Use Case e lascia al Domain layer la gestione della logica

Contiene le regole di bussiness pure.

Gestisce la persistenza della sessione.

Dependency Injection (1/2)

Problemi senza DI:

- **Dipendenze hardcoded:** impossibile sostituire un componente senza modificare il codice sorgente
- **Testing difficile:** non è possibile iniettare mock al posto delle dipendenze reali
- **Alto accoppiamento tra i layer:** un cambio in un componente si propaga a cascata

Vantaggi con DI:

- **Componenti sostituibili** senza toccare la business logic
- **Testing isolato** tramite mock (come dimostrato dai test con MagicMock)
- **Rispetto del Dependency Inversion Principle**: i layer dipendono da astrazioni, non da implementazioni concrete

Dependency Injection (2/2)

```python
from fastapi import Depends

from app.application.interfaces.session_service import ISessionService
from app.application.interfaces.use_cases.create_session import (
    ICreateSessionUseCase,
)
from app.application.use_cases.session.create_session import CreateSessionUseCase

from ..shared_dependencies import get_session_service


def get_create_session_use_case(
    service: ISessionService = Depends(get_session_service),
) -> ICreateSessionUseCase:
    return CreateSessionUseCase(service)
```

FastAPI costruisce e inietta le dipendenze tramite Depends

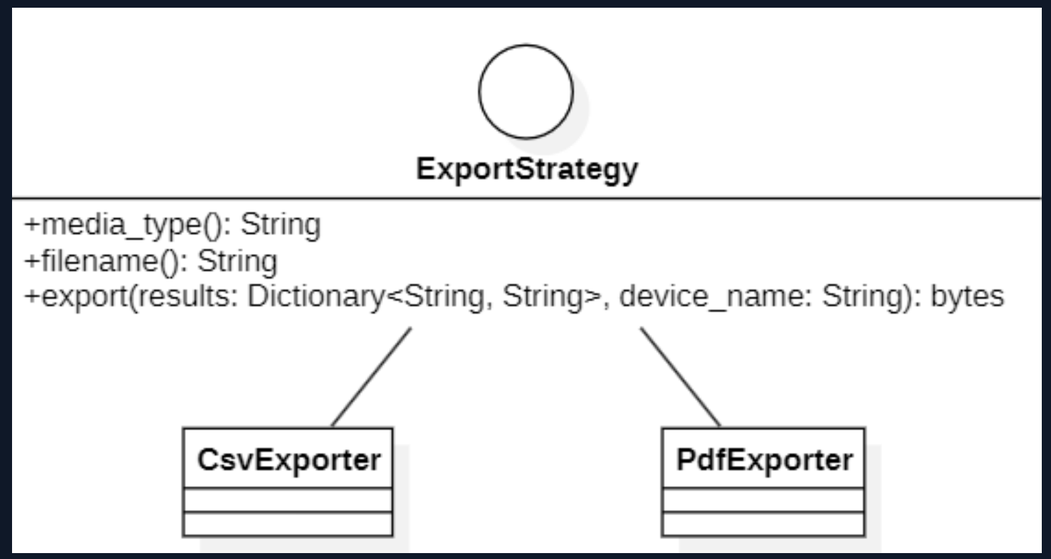
Nel controller le dipendenze arrivano già costruite:

```python
from fastapi import Depends
from fastapi_utils.cbv import cbv
from fastapi_utils.inferring_router import InferringRouter

from app.application.interfaces.use_cases.create_session import (
    ICreateSessionUseCase,
)

from .dependencies import (
    get_create_session_use_case,
)
from .schema import (
    SessionResponseSchema,
)

router = InferringRouter(prefix="/session", tags=["session"])

@cbv(router)
class SessionController:
    create_session_use_case: ICreateSessionUseCase = Depends(get_create_session_use_case)
    ...

    @router.post("/create_session_with_file", status_code=201)
    async def create_session_with_file(self, parameters) -> SessionResponseSchema:
        ...
        # code
        # result = await self.create_session_use_case.execute(CreateSessionRequest())
        # return SessionResponseSchema(result)
```

Strategy Pattern

Problemi:

- **Esportazione in formati multipli:** l'esportazione dei risultati può avvenire in CSV oppure in PDF.



Vantaggi con Strategy:

- **Interfaccia unica:** contenente i metodi implementati nello specifico da ogni classe derivata.
- **Possibilità di estensione** senza dover modificare l'architettura già esistente.

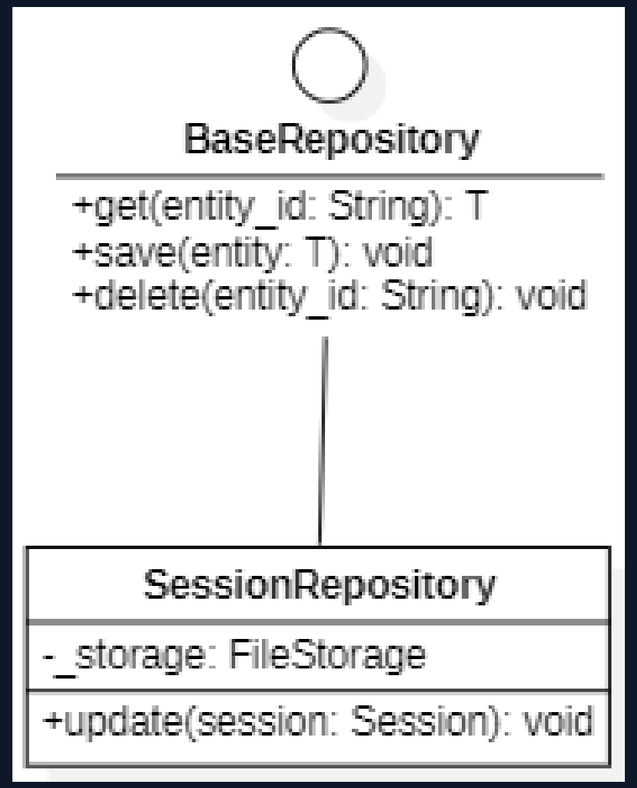
Repository Pattern

Vantaggi

- Separa la logica di dominio dalla persistenza
- Permette di testare senza filesystem reale (mock del storage)
- Centralizza la cache in memoria in un unico punto

Svantaggi senza

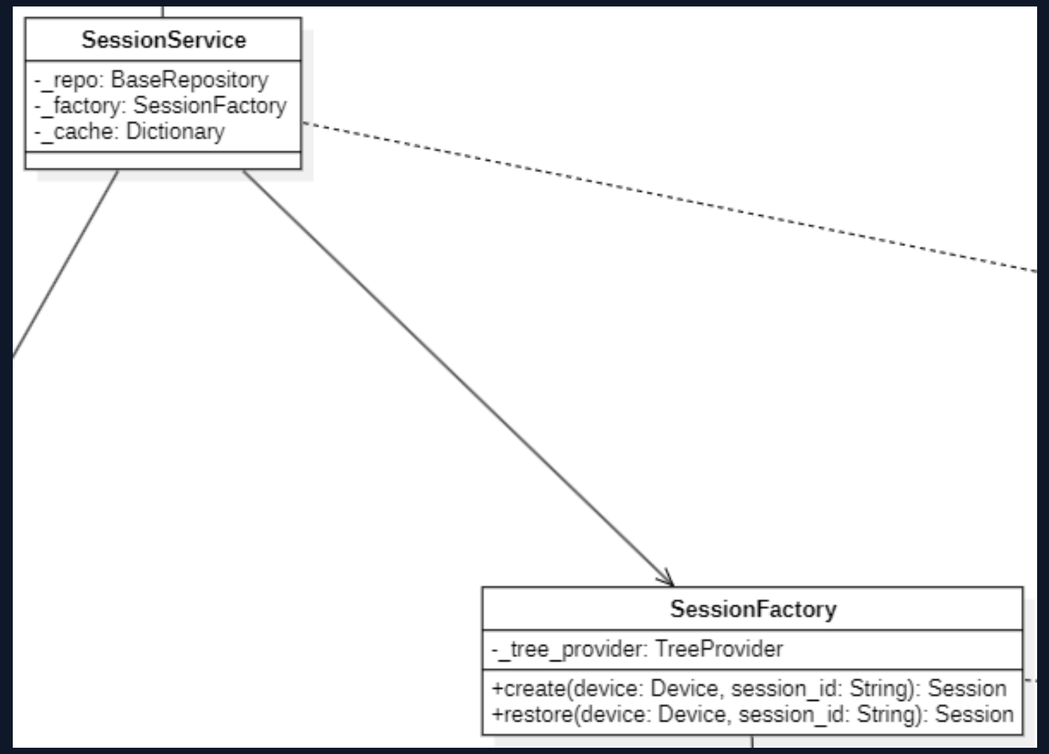
- Session e DecisionTree dovrebbero gestire direttamente JSON e path
- I test richiederebbero un filesystem reale
- Violazione del Single Responsibility Principle: le entità farebbero troppo



Factory Pattern

Vantaggi

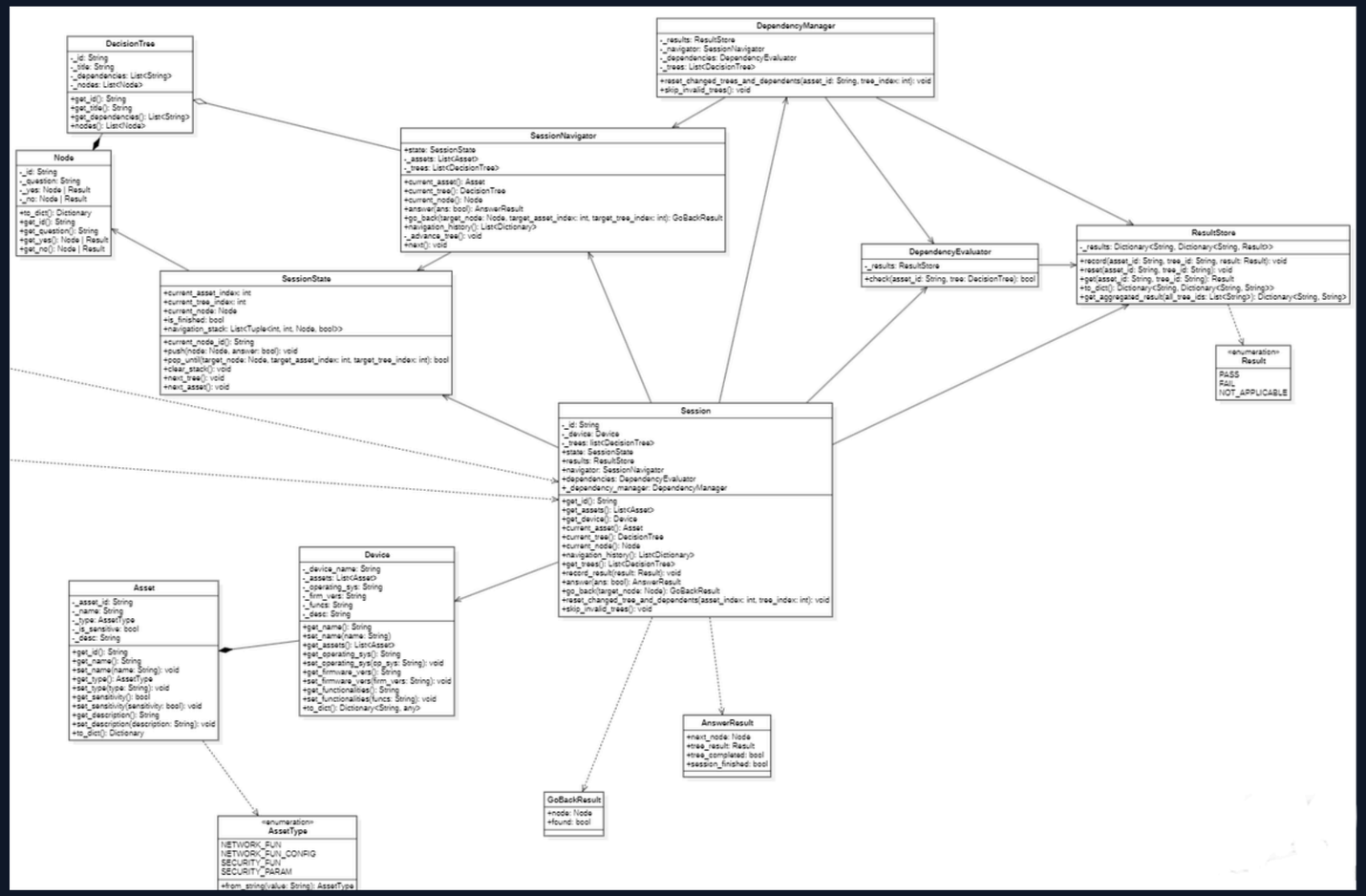
- **Centralizza la creazione di Session** che richiede molte dipendenze
- **Offre due metodi distinti** create() e restore() per i due casi d'uso principali
- Garantisce che ogni sessione creata abbia sempre tutte le dipendenze necessarie, **evitando stati inconsistenti**



Facade Pattern

Vantaggi

- **Espone un'interfaccia semplice** nascondendo la complessità interna delle classi utilizzate da Session.
- **Riduce il coupling** tra gli use case e i componenti interni — AnswerUseCase chiama solo session.answer() senza conoscere SessionNavigator



Frontend: design patterns (1/3)

- **Custom Hooks**

  Incapsulano la logica separandola dai componenti di presentazione e rendendoli più puliti.

```
const hasAccess = useRouteGuardAccess(routeConfig)
```

```
const actions = useHomeNavigation()
```

```
const { register, handleSubmit, resetForm, onCancel, errors } =
useDeviceForm()
```

- **Context Provider**

  Evita il prop drilling e permette di condividere dipendenze

```
createRoot(document.getElementById('root')).render(
    <StrictMode>
        <NotificationProvider>
            <AppRouter />
            <Toaster position="top-right" richColors />
        </NotificationProvider>
    </StrictMode>
)
```

```
const NotificationContext = createContext(null)

function NotificationProvider({ value, children }) {
    const services = value ? { ...defaultServices, ...value } :
    defaultServices
    return createElement(NotificationContext.Provider, { value:
    services }, children)
}
```

Frontend: design patterns (2/3)

- **Flux-like state management**

Lo stato condiviso è gestito con store separati per responsabilità e selector mirati; il flusso resta unidirezionale

```
useSessionStore.getState().setSessionId(sessionId)
useSessionStore.getState().setSessionUploaded(false)
useDeviceStore.getState().setDevice(deviceObj)
```

- selectors
  - DeviceStore.js
  - ResultStore.js
  - SessionStore.js
  - TreeStore.js
  - UIStore.js

```
import { create } from 'zustand'
import { devtools } from 'zustand/middleware'

const INITIAL_SESSION_STATE = {
    sessionId: null,
    pastHistory: [],
    futureHistory: [],
    isTestFinished: false,
    isResumeMode: false,
    isSessionUploaded: false,
    resultsPerAsset: null,
    currentNode: null,
    currentAssetIndex: 0,
    currentTreeIndex: 0,
}
```

- **Ports and Adapters**

Per evitare dipendenza diretta della logica applicativa da librerie tecniche specifiche.

```
export class IApiClient {
    constructor() {
        if (this.constructor === IApiClient) {
            throw new Error('IApiClient is an abstract class and cannot be
            instantiated.')
        }
    }

    get(...args) {
        void args
        throw new Error("Method 'get' not implemented.")
    }

    post(...args) {
        void args
        throw new Error("Method 'post' not implemented.")
    }

    delete(...args) {
        void args
        throw new Error("Method 'delete' not implemented.")
    }
}
```

```
import axios from 'axios'
import { IApiClient } from '@application/ports/IApiClient'

class AxiosApiClient extends IApiClient {
```

Atlas

Frontend: design patterns (3/3)

- **Facade**

  Ogni service espone operazioni ad alto livello (crea sessione, invia risposta, esporta risultati, carica alberi) nascondendo dettagli di chiamate HTTP, mapping dati e aggiornamento store.

```
class SessionService {
  async createSessionWithDevice(device) { ... }
  async sendAnswer(answer) { ... }
  async loadSessionFromFile(file) { ... }
  async saveAndExit() { ... }
}
export default new SessionService()
```